Topics Discussed — Day 3

1. Introduction to SQL

› 1.1 What is SQL?
› 1.2 What is a Database?
› 1.3 What is a Query?
› 1.4 SQL Server (MSSQL) and SSMS

2. SQL Language Categories

› DDL — Data Definition Language
› DML — Data Manipulation Language
› DQL — Data Query Language
› DCL — Data Control Language

3. DDL — Data Definition Language

› 3.1 CREATE DATABASE
› 3.2 CREATE TABLE & Data Types
› 3.3 ALTER TABLE (ADD / MODIFY column)
› 3.4 DROP TABLE
› 3.5 TRUNCATE TABLE

4. DML — Data Manipulation Language

› 4.1 INSERT INTO (Add Data)
› 4.2 SELECT (Read/Retrieve Data)
› 4.3 UPDATE (Modify Data)
› 4.4 DELETE (Remove Data)

5. SQL Data Types — Quick Reference

› INT, BIGINT, VARCHAR, NVARCHAR, CHAR
› DATE, DATETIME, DECIMAL, FLOAT, BIT

6. Real-Time Interview Questions & Answers

› 6.1 SQL Basics
› 6.2 DDL Questions
› 6.3 DML Questions
› 6.4 Scenario-Based Questions

7. Quick Revision Cheat Sheet

1. Introduction to SQL

1.1 What is SQL?

Definition

SQL stands for Structured Query Language. It is the standard language used to communicate with relational databases. You can use SQL to create databases, add data, update records, delete data, and retrieve (query) specific information.

Think of SQL like a language you use to talk to a database. Just like you use English to ask a librarian for a book, you use SQL to ask a database for data!

1.2 What is a Database?

Definition

A Database is an organized collection of data stored electronically. It stores data in tables — similar to Excel spreadsheets — with rows (records) and columns (fields).

Example: A school database may have tables like Students, Teachers, Courses, and Marks.

1.3 What is a Query?

Definition

A Query is a request or command sent to a database to perform an action — such as fetching, inserting, updating, or deleting data.

Example: "Give me the names of all students who scored above 80" — this is a query!

1.4 SQL Server (MSSQL) and SSMS

SQL Server (MSSQL)

Microsoft SQL Server (also called MSSQL) is a popular Relational Database Management System (RDBMS) made by Microsoft. It stores and manages data using SQL.

SSMS — SQL Server Management Studio

SSMS is a free graphical tool (software) provided by Microsoft that you use to connect to SQL Server, write SQL queries, and manage databases. Think of it as the "editor" for SQL.

■ Quick Tip

To start using SQL: Install SQL Server + SSMS → Connect to the Server → Create a Database → Start writing queries!

2. SQL Language Categories

SQL statements are grouped into categories based on what they do:

Category	Full Form	Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	INSERT, UPDATE, DELETE, SELECT
DQL	Data Query Language	SELECT
DCL	Data Control Language	GRANT, REVOKE

3. DDL — Data Definition Language

DDL statements define the structure of your database — they create, modify, or delete database objects like tables, databases, and views.

3.1 CREATE DATABASE

Definition

The CREATE DATABASE statement creates a new database. A database is a container that holds all your tables and data.

Syntax

```
CREATE DATABASE database_name;
```

Example

```
-- Create a new database for our institute
CREATE DATABASE FLM;
```

Output: A new database named "FLM" is created in SQL Server.

■ Quick Tip

Always use meaningful names for your database. Avoid spaces — use underscores (FLM_School) instead.

3.2 CREATE TABLE

Definition

The CREATE TABLE statement creates a new table inside a database. A table is made up of columns (fields) and rows (records). You must define each column name and its data type.

Common Data Types

- INT / INTEGER — Whole numbers (e.g., 1, 100, 2024)
- BIGINT — Large whole numbers (e.g., 1234567890)
- VARCHAR(n) — Variable-length text, up to n characters (e.g., "John")
- NVARCHAR(n) — Unicode text (supports all languages)
- DATE — Date values (e.g., 2024-01-15)
- DECIMAL(p, s) — Numbers with decimals (e.g., 99.99)

Syntax

```
CREATE TABLE table_name
(
column1_name datatype,
column2_name datatype,
column3_name datatype
);
```

Example

```
-- Create a Students table
CREATE TABLE flmstudents
(
studentid INT,
studentname VARCHAR(100),
emailid VARCHAR(100),
address NVARCHAR(250)
);
```

```
Result: Table "flmstudents" created successfully.

+-----------+------------+----------+----------+
| studentid | studentname | emailid | address |
+-----------+------------+----------+----------+
| INT | VARCHAR(100)| VARCHAR | NVARCHAR |
+-----------+------------+----------+----------+
```

■ Quick Tip

The table is empty when first created — it has structure (columns) but no data (rows) yet.

3.3 ALTER TABLE

Definition

The ALTER TABLE statement modifies an existing table structure. You can add new columns, change column data types, or remove columns — without deleting the table or its data.

ALTER TABLE — ADD a new column

```
ALTER TABLE table_name
ADD column_name datatype;
```

Example: Add emailid column

```
-- Adding an email column to flmstudents
ALTER TABLE flmstudents
ADD emailid VARCHAR(100);
```

```
Result: Column 'emailid' added to flmstudents table.
```

ALTER TABLE — MODIFY an existing column

```
ALTER TABLE table_name
ALTER COLUMN column_name new_datatype;
```

Example: Increase emailid column size

```
-- Change emailid from VARCHAR(100) to VARCHAR(150)
ALTER TABLE flmstudents
ALTER COLUMN emailid VARCHAR(150);
```

```
Result: Column 'emailid' changed from VARCHAR(100) to VARCHAR(150).
```

■ Quick Tip

Use ALTER TABLE when you need to change a table's structure without losing any existing data.

3.4 DROP TABLE

Definition

The DROP TABLE statement permanently deletes a table AND all its data from the database. This action is irreversible — once dropped, you cannot recover the table or its data.

Syntax

```
DROP TABLE table_name;
```

Example

```
-- Permanently delete the flmstudents table
DROP TABLE flmstudents; -- WARNING: All data is lost!
```

```
Result: Table 'flmstudents' and all its data permanently deleted.
```

■■ WARNING

DROP TABLE is a dangerous command. Always double-check before running it. In production databases, always take a backup first!

3.5 TRUNCATE TABLE

Definition

The TRUNCATE TABLE statement removes ALL rows (data) from a table but KEEPS the table structure (columns remain). It is faster than DELETE because it does not log each row deletion.

Syntax

```
TRUNCATE TABLE table_name;
```

Example

```
-- Remove all data but keep the table structure
TRUNCATE TABLE flmstudents;
```

```
Result: All rows deleted.
Table 'flmstudents' still exists but is now empty.
```

Command	Removes Data?	Removes Table?
TRUNCATE TABLE	YES (all rows)	NO — keeps structure
DROP TABLE	YES (all rows)	YES — table is gone

4. DML — Data Manipulation Language

DML statements are used to work with the data inside tables. After you create a table using DDL, you use DML to add, update, delete, and retrieve data.

4.1 INSERT INTO — Add Data

Definition

The INSERT INTO statement adds new rows (records) into a table. You specify the table name, the columns, and the values to insert.

Syntax

```
-- Insert with specific columns
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);

-- Insert into all columns
INSERT INTO table_name
VALUES (value1, value2, value3);
```

Example: Add students

```
INSERT INTO flmstudents (studentid, studentname, emailid, address)
VALUES (1, 'Rahul Sharma', 'rahul@gmail.com', 'Hyderabad');

INSERT INTO flmstudents (studentid, studentname, emailid, address)
VALUES (2, 'Priya Singh', 'priya@gmail.com', 'Mumbai');

INSERT INTO flmstudents (studentid, studentname, emailid, address)
VALUES (3, 'Amit Kumar', 'amit@gmail.com', 'Delhi');
```

```
Result: 3 rows inserted into flmstudents.

+-----------+------------+-----------------+----------+
| studentid | studentname | emailid | address |
+-----------+------------+-----------------+----------+
| 1 | Rahul Sharma| rahul@gmail.com | Hyderabad |
| 2 | Priya Singh | priya@gmail.com | Mumbai |
| 3 | Amit Kumar | amit@gmail.com | Delhi |
+-----------+------------+-----------------+----------+
```

■ Quick Tip

Text/string values must be written inside single quotes: 'Rahul'. Numbers (INT) do not need quotes: 1, 2, 3.

4.2 SELECT — Read / Retrieve Data

Definition

The SELECT statement is used to retrieve data from a table. It is the most frequently used SQL command. You can select all columns or specific columns.

Syntax

```
-- Select all columns
SELECT * FROM table_name;

-- Select specific columns
SELECT column1, column2 FROM table_name;

-- Select with a condition
SELECT * FROM table_name WHERE condition;
```

Examples

```
-- Get all students
SELECT * FROM flmstudents;

-- Get only names and emails
SELECT studentname, emailid FROM flmstudents;

-- Get students from Hyderabad only
SELECT * FROM flmstudents WHERE address = 'Hyderabad';
```

```
+-----------+-------------+-----------------+-----------+
| studentid | studentname | emailid | address |
+-----------+-------------+-----------------+-----------+
| 1 | Rahul Sharma| rahul@gmail.com | Hyderabad |
| 2 | Priya Singh | priya@gmail.com | Mumbai |
| 3 | Amit Kumar | amit@gmail.com | Delhi |
+-----------+-------------+-----------------+-----------+
```

4.3 UPDATE — Modify Existing Data

Definition

The UPDATE statement modifies existing data in a table. You use WHERE clause to specify which rows to update. Without WHERE, ALL rows will be updated!

Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Example: Update a student's address

```
-- Update Rahul's address
UPDATE flmstudents
SET address = 'Bangalore'
WHERE studentid = 1;

-- Update multiple columns at once
UPDATE flmstudents
SET emailid = 'rahul.sharma@gmail.com', address = 'Chennai'
WHERE studentname = 'Rahul Sharma';
```

```
Result: 1 row updated.

+-----------+-------------+-----------------------+---------+
| studentid | studentname | emailid | address |
+-----------+-------------+-----------------------+---------+
| 1 | Rahul Sharma| rahul.sharma@gmail.com | Chennai | <- Updated
| 2 | Priya Singh | priya@gmail.com | Mumbai |
| 3 | Amit Kumar | amit@gmail.com | Delhi |
+-----------+-------------+-----------------------+---------+
```

■■ IMPORTANT

Always use the WHERE clause with UPDATE. Without WHERE, every single row in the table will be updated!

4.4 DELETE — Remove Data

Definition

The DELETE statement removes specific rows from a table based on a condition. Unlike DROP TABLE, DELETE only removes rows — the table structure stays. Always use WHERE to avoid deleting all data.

Syntax

```
DELETE FROM table_name
WHERE condition;
```

Example

```sql
-- Delete one specific student
DELETE FROM flmstudents
WHERE studentid = 3;

-- Delete all students from Delhi
DELETE FROM flmstudents
WHERE address = 'Delhi';
```

```
Result: 1 row deleted (studentid = 3).

+-----------+------------+-----------------+----------+
| studentid | studentname | emailid | address |
+-----------+------------+-----------------+----------+
| 1 | Rahul Sharma| rahul@gmail.com | Hyderabad |
| 2 | Priya Singh | priya@gmail.com | Mumbai |
+-----------+------------+-----------------+----------+
(Amit Kumar's record has been removed)
```

DDL vs DML — Quick Comparison

Feature	DDL	DML
Purpose	Define / modify structure	Manipulate data
Commands	CREATE, ALTER, DROP, TRUNCATE	INSERT, SELECT, UPDATE, DELETE
Affects	Table structure (columns)	Table data (rows)
Rollback	Not possible (auto-commit)	Possible (transaction)

5. SQL Data Types — Quick Reference

When creating tables, every column needs a data type. Here are the most common ones:

Data Type	Description	Example Usage	Sample Values
`INT`	Whole numbers	studentid INT	1, 42, 1000
`BIGINT`	Very large whole numbers	phonenumber BIGINT	9876543210
`VARCHAR(n)`	Text up to n chars	name VARCHAR(100)	'John', 'Rahul'
`NVARCHAR(n)`	Unicode/multilingual text	address NVARCHAR(250)	'Ahmed', '■■■■■'
`CHAR(n)`	Fixed-length text	gender CHAR(1)	'M', 'F'
`DATE`	Date only	dob DATE	'2000-05-15'
`DATETIME`	Date and time	created_at DATETIME	'2024-01-15 10:30'
`DECIMAL(p,s)`	Precise decimals	marks DECIMAL(5,2)	99.50, 78.25
`FLOAT`	Floating-point numbers	percentage FLOAT	85.6, 92.3
`BIT`	Boolean (0 or 1)	isactive BIT	0 (false), 1 (true)

6. Real-Time Interview Questions & Answers

These questions are commonly asked in SQL interviews for fresher and junior developer roles. Study them carefully!

6.1 SQL Basics

Q: What is SQL? Why is it used?

A: SQL (Structured Query Language) is a standard language used to interact with relational databases. It is used to create, read, update, and delete data stored in database tables. SQL provides a simple and efficient way to manage large amounts of data.

Q: What is the difference between SQL and MySQL/SQL Server?

A: SQL is a language — a set of commands. MySQL and SQL Server (MSSQL) are database management systems (DBMS) that use SQL as their query language. Think of SQL as English, and MySQL/SQL Server as different books written in English.

Q: What is a database? What is a table?

A: A database is an organized collection of data stored electronically. A table is a database object that stores data in rows and columns — similar to a spreadsheet. A database can contain many tables.

Q: What is a primary key?

A: A primary key is a column (or set of columns) that uniquely identifies each row in a table. It must be unique and cannot be NULL. For example, studentid in flmstudents can be a primary key since no two students share the same ID.

Q: What is the difference between WHERE and HAVING?

A: WHERE filters rows BEFORE grouping (works on individual rows). HAVING filters groups AFTER a GROUP BY clause. Example: WHERE studentid > 1 filters rows; HAVING COUNT(*) > 2 filters groups.

6.2 DDL Questions

Q: What are DDL commands? Name them.

A: DDL stands for Data Definition Language. DDL commands define or modify the structure of database objects. The main DDL commands are: CREATE (create database/table), ALTER (modify structure), DROP (permanently delete), and TRUNCATE (remove all rows but keep structure).

Q: What is the difference between DROP and TRUNCATE?

A: DROP TABLE permanently deletes the table AND all its data — the table no longer exists. TRUNCATE TABLE removes only the data (all rows) but keeps the table structure intact. DROP is more destructive; TRUNCATE just empties the table.

Q: What is the difference between TRUNCATE and DELETE?

A: TRUNCATE removes ALL rows at once and is faster (does not log individual row deletions). DELETE can remove specific rows using a WHERE clause and is logged (supports rollback). TRUNCATE cannot have a WHERE clause.

Q: Can you recover data after a DROP TABLE command?

A: No — in most cases, DROP TABLE is irreversible. Once executed, the table and all its data are permanently removed. Always back up data before using DROP TABLE in production.

6.3 DML Questions

Q: What are DML commands? Name them.

A: DML stands for Data Manipulation Language. DML commands work with data inside tables. The main DML commands are: INSERT (add new rows), SELECT (read/retrieve data), UPDATE (modify existing data), and DELETE (remove rows).

Q: What happens if you run UPDATE without a WHERE clause?

A: ALL rows in the table will be updated with the new values. This is a common mistake that can corrupt all your data. Always specify WHERE to target specific rows.

Q: What happens if you run DELETE without a WHERE clause?

A: ALL rows in the table will be deleted. The table structure will remain, but it will be completely empty. Always use WHERE with DELETE to remove only specific rows.

Q: How do you insert multiple rows at once?

A: In SQL Server, you can insert multiple rows in one INSERT statement. Example: INSERT INTO flmstudents VALUES (1, 'Rahul', 'rahul@mail.com', 'Hyd'), (2, 'Priya', 'priya@mail.com', 'Mum'); — More efficient than separate INSERT statements.

Q: What is SELECT * FROM table_name?

A: SELECT * means "select all columns". FROM table_name specifies which table to read. So SELECT * FROM flmstudents retrieves all columns and rows from flmstudents. In professional code, it is better to specify only the columns you need.

6.4 Scenario-Based Questions

Q: You accidentally deleted all records with DELETE. Can you recover them?

A: If the DELETE was inside a transaction (BEGIN TRANSACTION), run ROLLBACK to undo it. If no transaction was used, the data is lost unless you have a database backup. Always (1) use WHERE with DELETE, (2) take backups, and (3) use transactions for critical operations.

Q: What is the difference between INT and BIGINT?

A: INT stores whole numbers from approximately -2.1 billion to +2.1 billion (32-bit). BIGINT stores much larger numbers (64-bit, up to about 9.2 quadrillion). Use INT for most IDs; use BIGINT for phone numbers or very large financial values.

Q: When would you use VARCHAR vs CHAR?

A: Use VARCHAR(n) when text length varies — like names or email addresses. It only uses as much space as needed. Use CHAR(n) when the length is always fixed — like gender code (M/F) or country code (IN, US). CHAR pads with spaces if shorter.

Q: How do you create a database and then create a table in it?

A: Step 1: CREATE DATABASE FLM; — creates the database. Step 2: USE FLM; — switches to that database. Step 3: CREATE TABLE flmstudents (...); — creates a table inside FLM. Always make sure you are in the right database before creating tables.

7. Quick Revision Cheat Sheet

```
-- ========== DATABASE ==========
CREATE DATABASE FLM; -- Create database
USE FLM; -- Switch to database

-- ========== DDL ==========
CREATE TABLE flmstudents ( -- Create table
studentid INT,
studentname VARCHAR(100),
emailid VARCHAR(100),
address NVARCHAR(250)
);

ALTER TABLE flmstudents -- Add column
ADD phonenumber VARCHAR(15);

ALTER TABLE flmstudents -- Modify column
ALTER COLUMN emailid VARCHAR(200);

TRUNCATE TABLE flmstudents; -- Delete all rows (keep table)
DROP TABLE flmstudents; -- Delete table permanently

-- ========== DML ==========
INSERT INTO flmstudents -- Add row
(studentid, studentname, emailid, address)
VALUES (1, 'Rahul', 'rahul@mail.com', 'Hyderabad');

SELECT * FROM flmstudents; -- Read all data
SELECT studentname FROM flmstudents -- Read with filter
WHERE address = 'Hyderabad';

UPDATE flmstudents -- Modify row
SET address = 'Mumbai'
WHERE studentid = 1;

DELETE FROM flmstudents -- Remove row
WHERE studentid = 1;
```